

Programação para Dispositivos Móveis I

Prof. Marcos Vasconcelos de Oliveira

Marcos.vasconcelos4@fatec.sp.gov.br



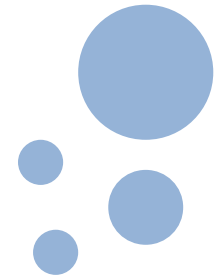
Aula 3 – Conceitos Básicos – Linguagem Kotlin

Prof. Marcos Vasconcelos de Oliveira

Marcos.vasconcelos4@fatec.sp.gov.br



Instalando o IntelliJ IDEA



- <https://www.jetbrains.com/idea/download>



Version: 2020.2
Build: 202.6397.94
27 July 2020

[Release notes](#)[System requirements](#)[Installation Instructions](#)[Other versions](#)

Download IntelliJ IDEA

[Windows](#) [Mac](#) [Linux](#)

Ultimate

For web and enterprise development

Download

.exe ▼

Free 30-day trial

Community

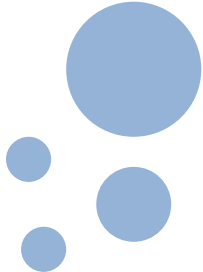
For JVM and Android development

Download

.exe ▼

Free, open-source

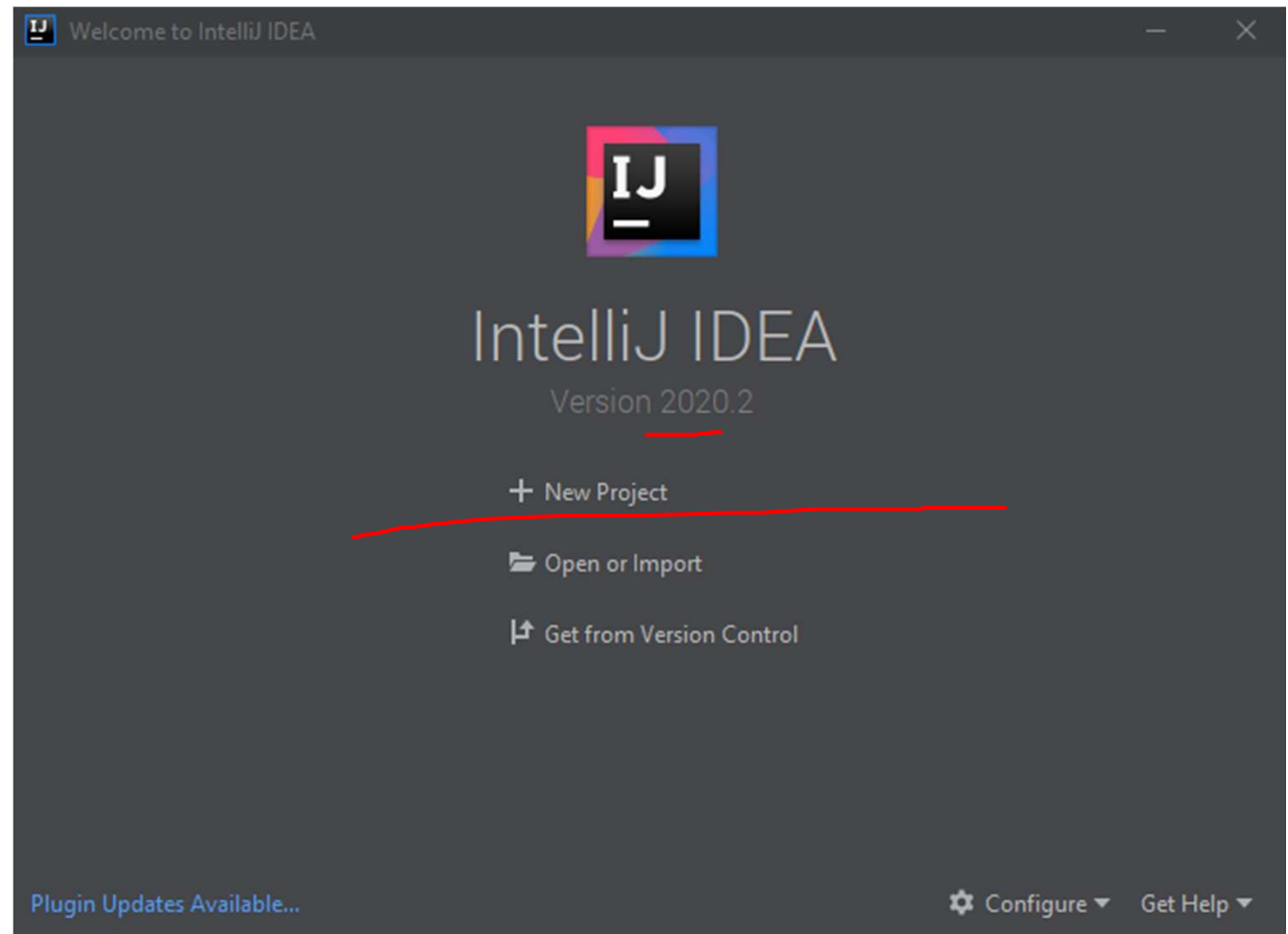
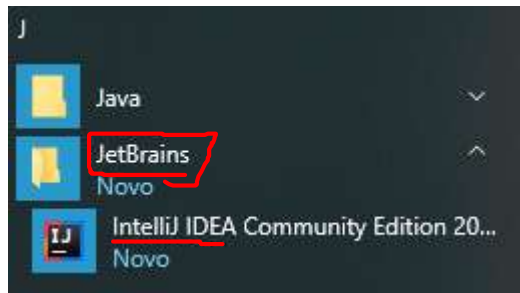
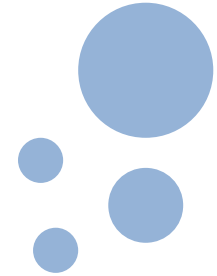
	IntelliJ IDEA Ultimate	IntelliJ IDEA Community Edition ⓘ
Java, Kotlin, Groovy, Scala	✓	✓
Android ⓘ	✓	✓
Maven, Gradle, sbt	✓	✓
Git, SVN, Mercurial	✓	✓
Debugger	✓	✓
Profiling tools ⓘ	✓	✗

- 
- 
- Kotlin compila para Bitcode e o Java também.
 - Bitcode será interpretado pela máquina virtual
do Java

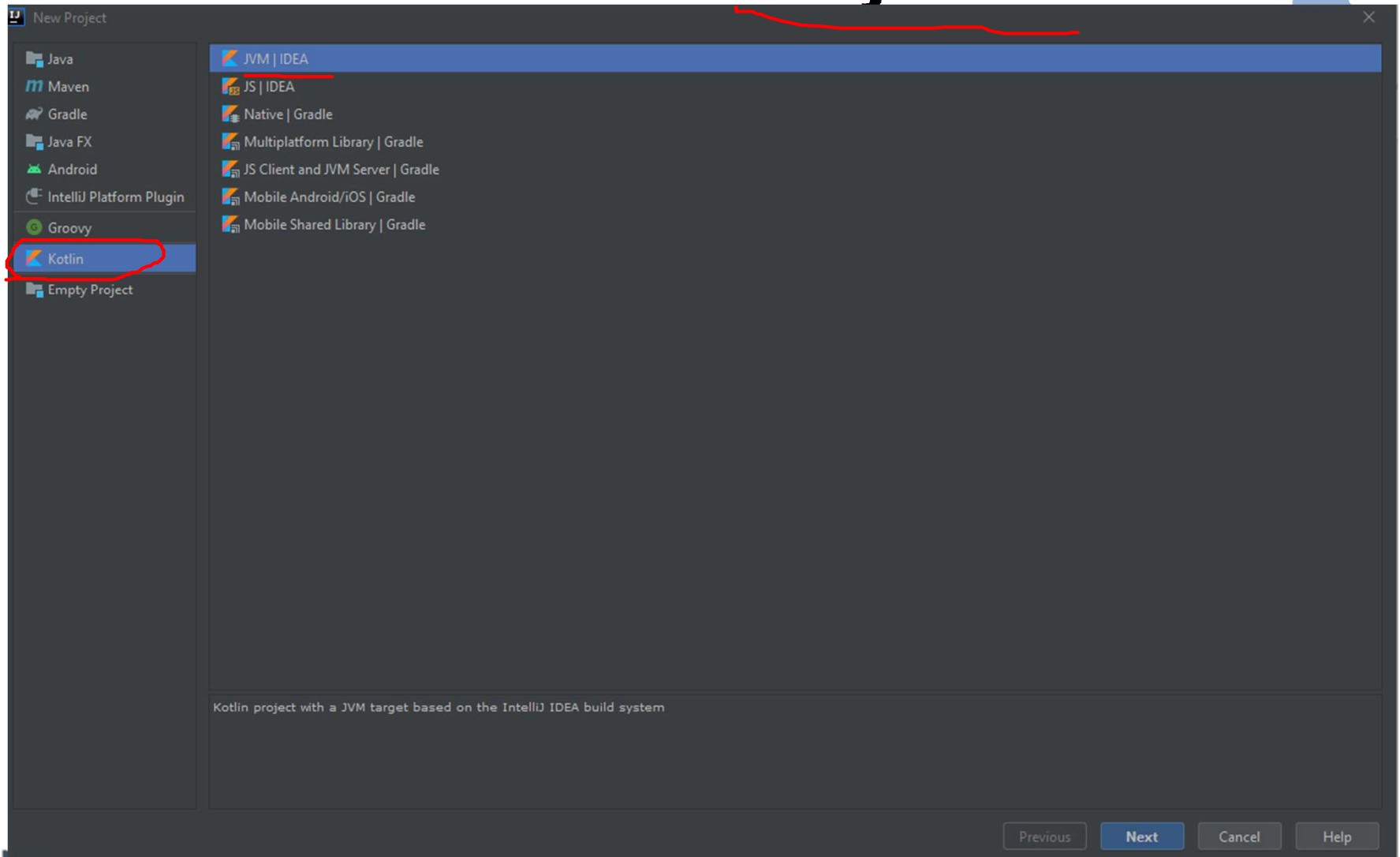


Kotlin

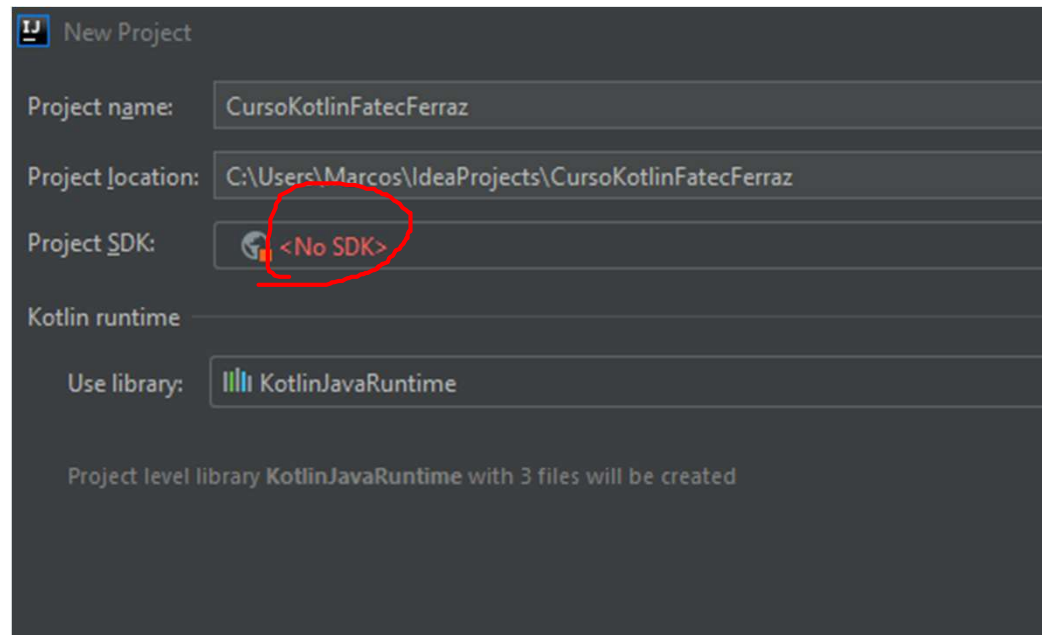
Abrindo o IntelliJ IDEA



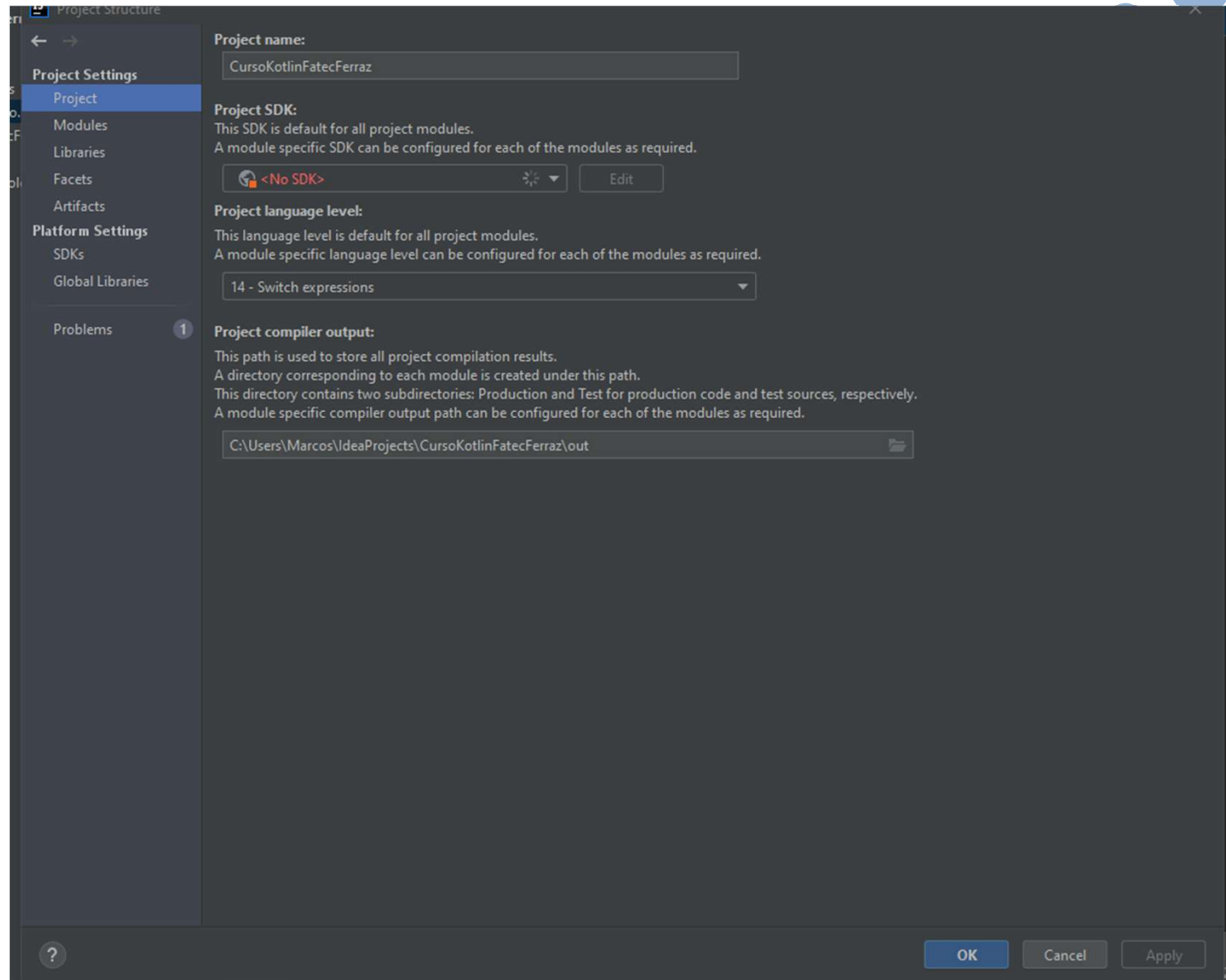
Criando um Projeto



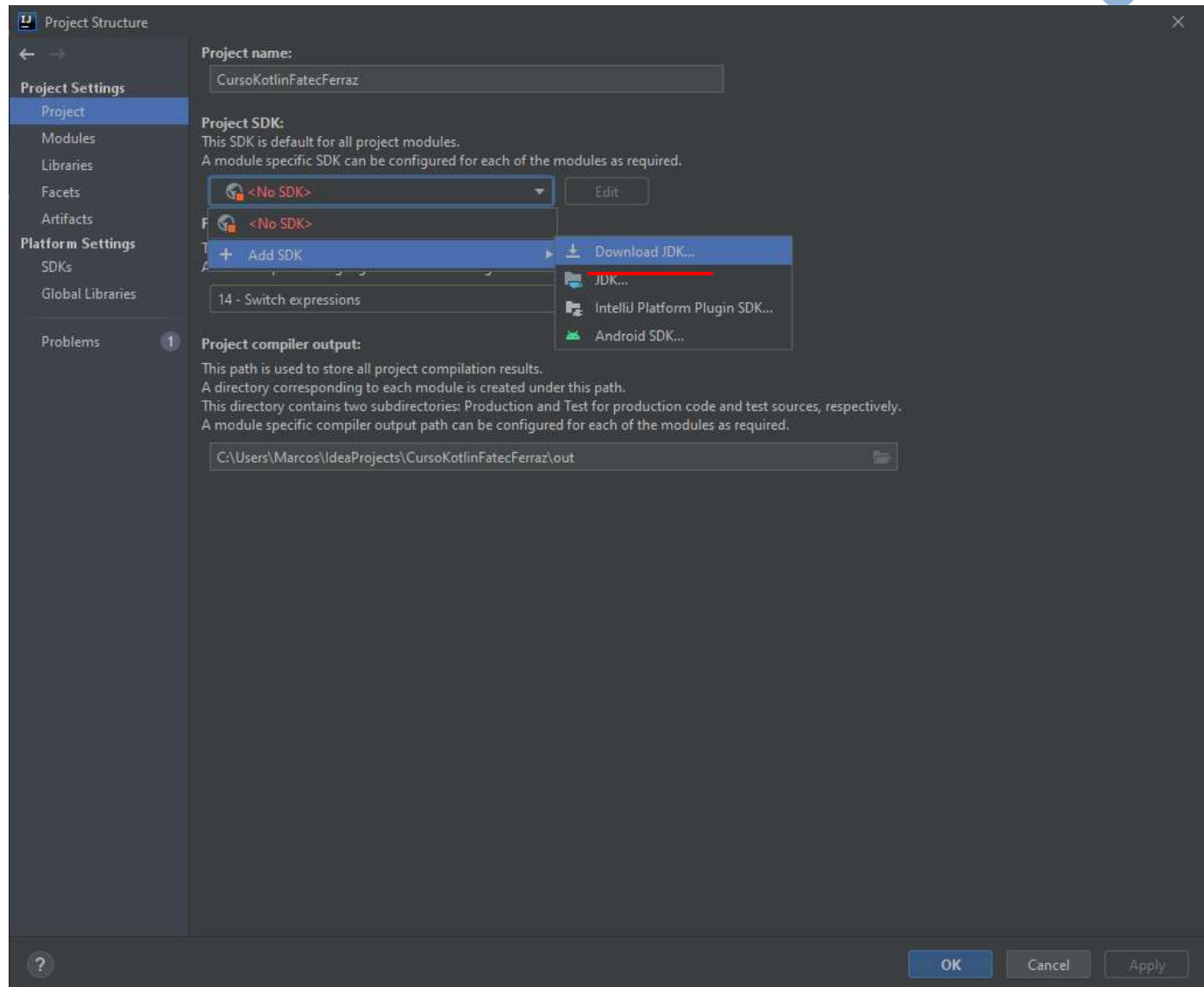
SDK do Java Não Instalado



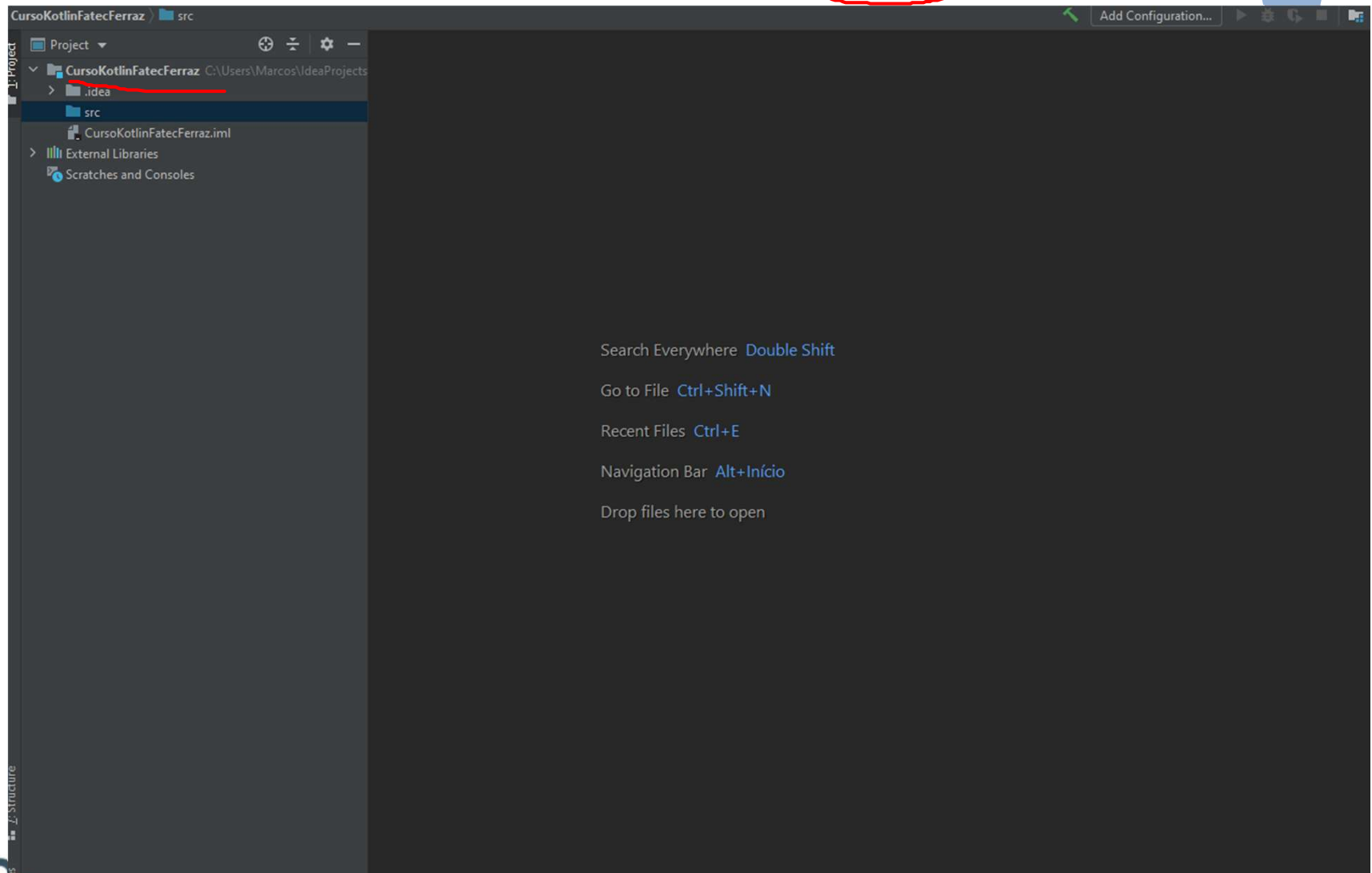
SDK do Java Não Instalado



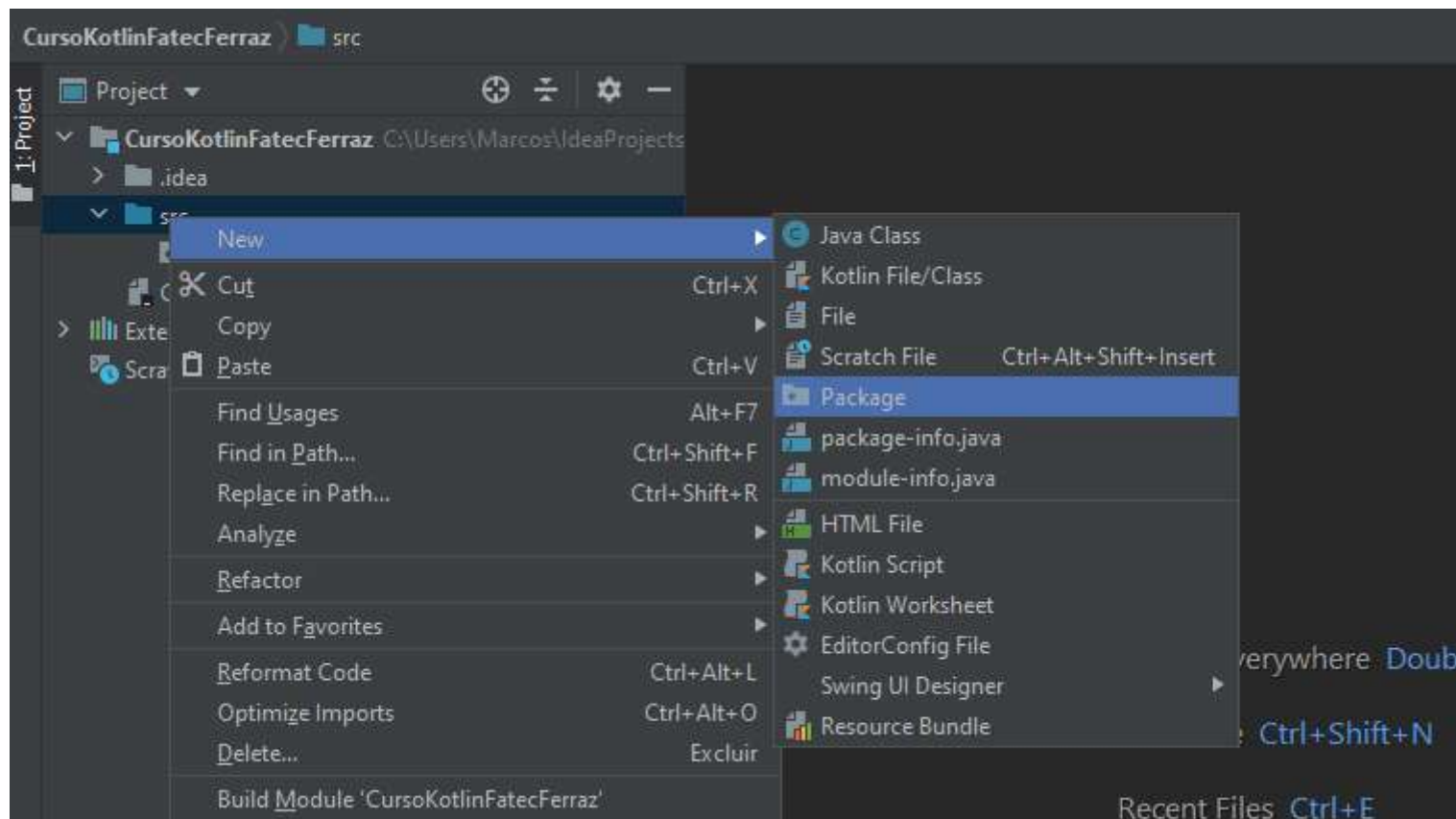
SDK do Java Não Instalado



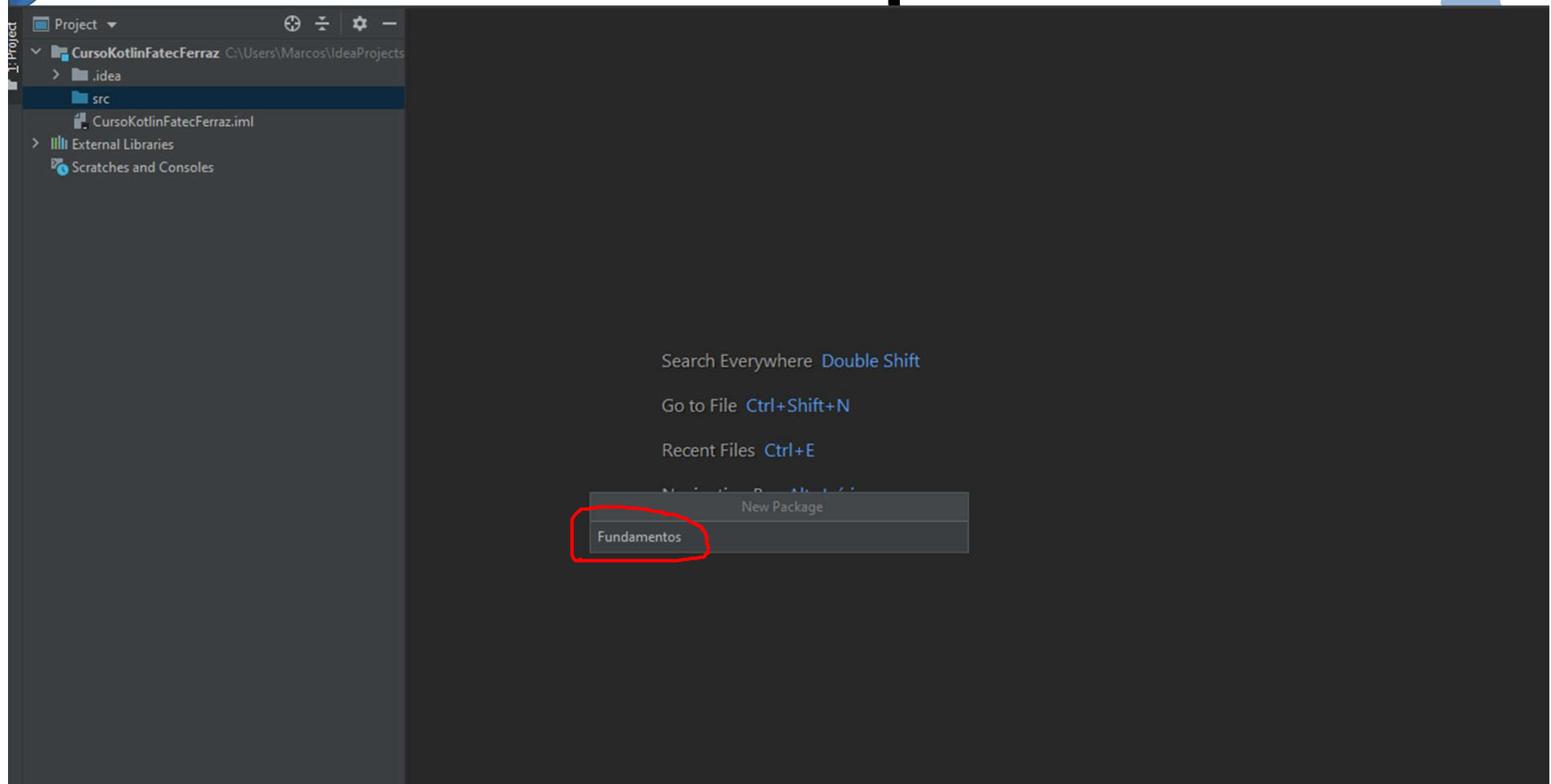
Criando um pacote



Criando um pacote

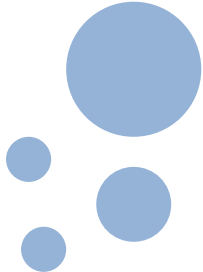


Criando um pacote





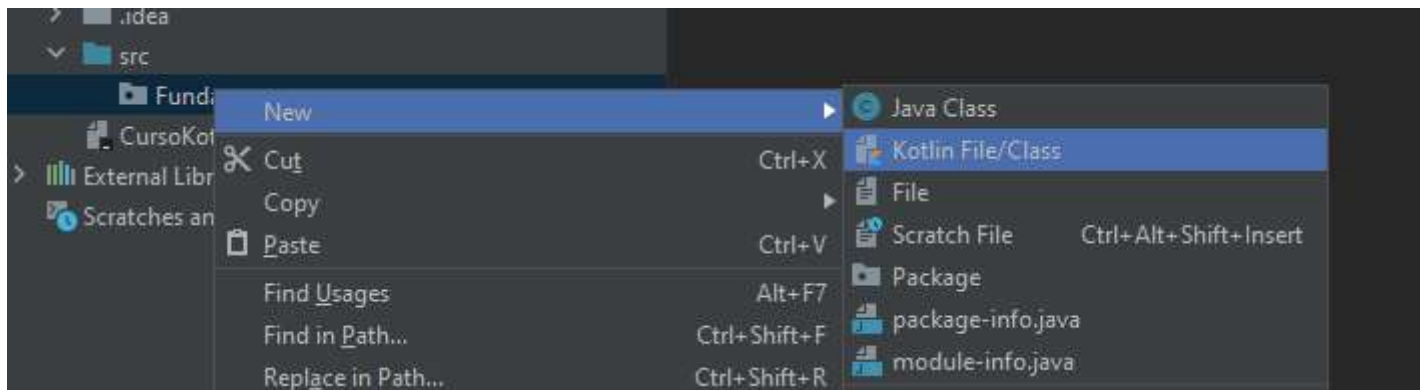
Criando um pacote



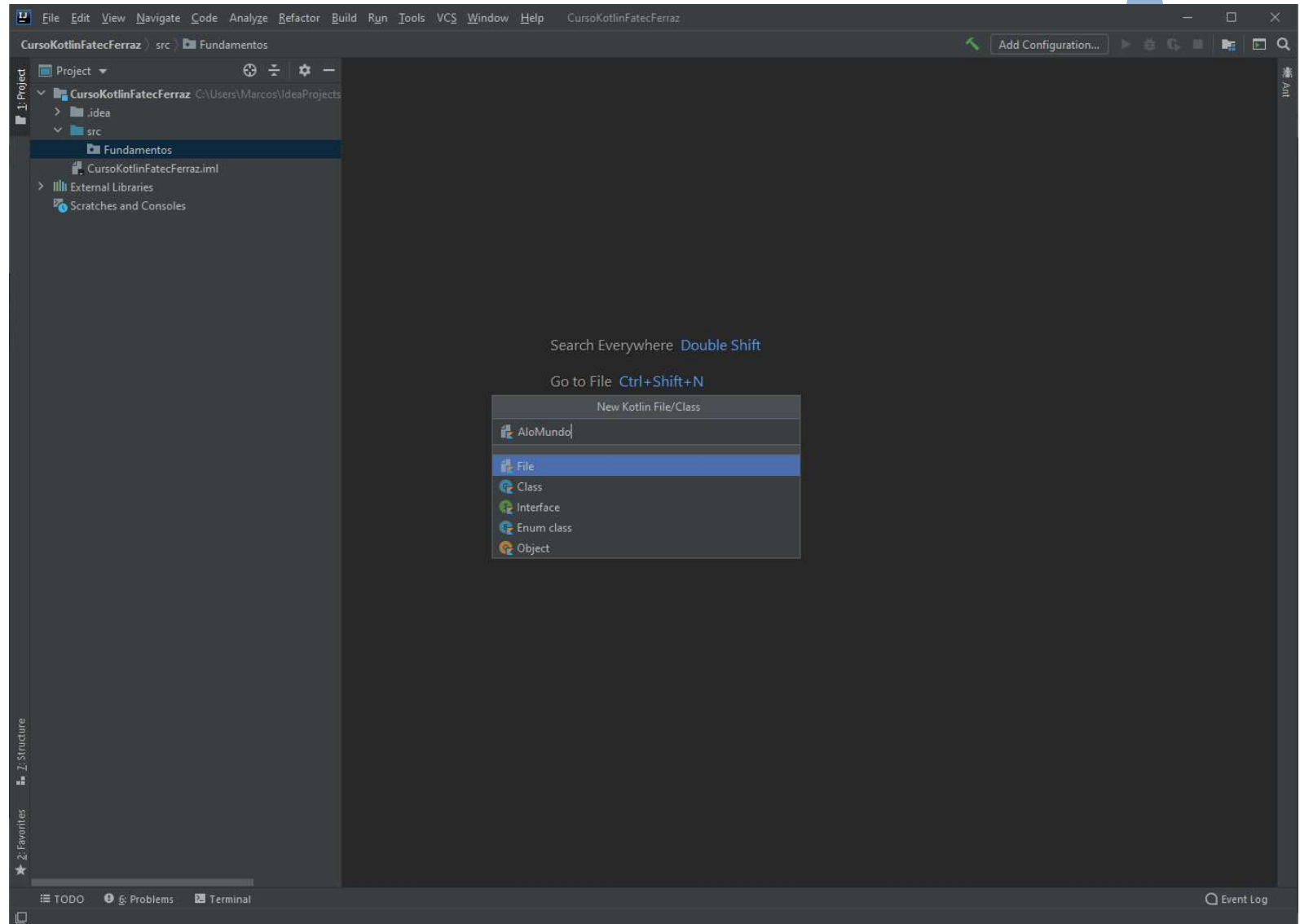
- Packages = pacote = “ideia de pasta de arquivos”

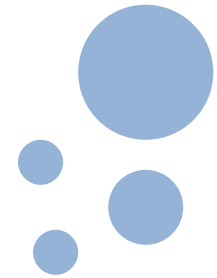


Criando um arquivo Kotlin



Criando um arquivo Kotlin

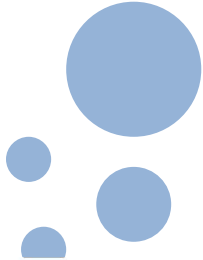




- Porta de entrada é o método Main
- Main + tab



Kotlin



Introdução

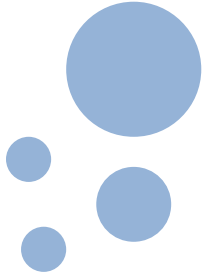
Uma linguagem de programação de tipagem estática, concisa, criada pela JetBrains e que está em constante desenvolvimento. É totalmente interoperável com Java e permite também a criação de aplicações web e nativas.

Kotlin é compatível com a JVM e pode ser usada no Android e na web com Spring ou qualquer outro framework Java. No Android, em especial, diversas alterações na API original foram distribuídas através da biblioteca KTX para que programadores Kotlin ficassem mais à vontade ao utilizar os recursos dessa linguagem





Linguagem Kotlin



- Desenvolvedores Kotlin
 - Podemos definir Kotlin como uma linguagem de programação pragmática que combina os paradigmas de Orientação a Objetos e Programação Funcional com foco em interoperabilidade, segurança, clareza e suporte a ferramentas.
- Equipe Android
 - Uma linguagem expressiva, concisa e poderosa.
- A linguagem foi criada pela JetBrains em 2011
- 2016 ➔ Versão 1.0
- Em 2017, Evento Google I/O, foi feito o anúncio de que Kotlin é oficialmente a linguagem para desenvolvimento Android
- Principais pontos da linguagem: interoperabilidade, concisão e proteção contra nulos.

Estrutura básica

```
fun main() {  
    println("Hello, world!")  
}
```

fun - palavra reservada para declarar uma função.

main() – nome da função.

{} – delimita o corpo da função, ou seja, as instruções que devem ser realizadas pela função.

println() – função para exibir conteúdo (saída padrão).



print e println

print() – função para exibir conteúdo (saída padrão) **sem** quebra de linha no final da string.

println() – função para exibir conteúdo (saída padrão) **com** quebra de linha no final da string.

\n – quebra de linha



The screenshot shows the 'Playground Kotlin' interface. At the top, it says 'Teste o Kotlin e pratique o que você aprendeu até agora. Digite seu código na janela abaixo e clique no botão para executá-lo.' Below this is a text area containing the following Kotlin code:

```
fun main() {  
    println("Hello, world!")  
    print("-----")  
    print("Bem vindos")  
    print("\nProgramação Kotlin")  
}
```

At the bottom of the interface, the output is displayed:

```
Hello, world!  
-----Bem vindos  
Programação Kotlin
```

At the bottom right of the output area, it says 'Target platform: JVM Running on Kotlin v. 1.5.21'.

Variável e Constante

- ✓ Representa um valor ou expressão (na memória)
- ✓ É declarada com a palavra-chave **var** | **val** seguida do nome
- ✓ O tipo da variável pode ser especificado ou não depois do nome

Exemplos:

```
val nome = "Miguel"           // inferência de tipo  
var nome: String = "Gabriel" // declaração do tipo  
var nome: String  
nome = "Tiago"
```

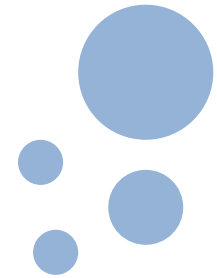
- ✓ **val** (de valor) representa uma referência imutável, ou seja, seu estado não pode ser modificado após ser criado (final em Java)
- ✓ **var** (de variável) representa uma referência mutável, ou seja, o valor da variável pode ser alterado após ser criada (variável regular em Java)

Tipos de Dados

- ✓ String (texto)
- ✓ Char (caracteres)
- ✓ Boolean (valores lógicos true ou false)
- ✓ Byte (pode ter valores de -128 a 127)
- ✓ Short (pode ter valores de -32768 a 32767)
- ✓ Int (pode ter valores de -231 a 231-1)
- ✓ Long (pode ter valores de -263 a 263-1)
- ✓ Float (valor real de precisão única)
- ✓ Double (valor real de precisão dupla)
- ✓ Number (pode ser usado quando não tivermos certeza de qual valor numérico será atribuído a uma variável durante o programa)

Conversões de Tipos

- ✓ toByte()
- ✓ toShort()
- ✓ toInt()
- ✓ toLong()
- ✓ toFloat()
- ✓ toDouble()
- ✓ toChar()



Operadores Aritméticos

Operador	Significado
+ 	Adição (também usado para concatenação de string)
-	Operador de Subtração
*	Operador de Multiplicação
/	Operador de Divisão
%	Operador Módulo

Prefixo Unário e Operadores de Incremento/Decremento

Operador	Significado	Expressão
+	Adição unária	+a
-	Menos unário (inverte o sinal)	-a
!	não (inverte o valor)	!a
++	Incremento: aumenta o valor em 1	++a
--	Decremento: diminui o valor em 1	--a

Operadores Relacionais

Operador	Significado	Expressão
>	Maior que	$a > b$
<	Menor que	$a < b$
>=	Maior ou igual a	$a \geq b$
<=	Menor ou igual a	$a \leq b$
==	É igual a	$a == b$
!=	Não é igual a	$a != b$

Operadores Lógicos

Operador	Descrição	Expressão
	true <u>se qualquer uma</u> das expressões booleanas forem true	(a>b) (a<c)
&&	true <u>se todas</u> as expressões booleanas forem true	(a>b) && (a<c)

Operador in

- ✓ Utilizamos o operador in para iterar sobre intervalos.
- ✓ Podemos utilizar o operador in para verificar se um valor está presente em um intervalo de valores. 🖱️
- ✓ Podemos verificar também se um valor não está presente em um intervalo de valores utilizando o operador com a negação (!in).

Operador	Expressão
in	a em b
!in	a !em b

Matrizes

Uma matriz é um contêiner que contém dados (valores) de um único tipo e são representadas pela classe Array.

Esta classe possui as funções get, set, size, propriedade e algumas outras funções úteis.

Exemplos:

```
var Array1 = arrayOf(33,25,7,13,8)
var Array2 = arrayOf<Int>(22,9,41,4,18)
var Array3 = arrayOf<String>("Léo","Ana","Luana")
var Array4 = arrayOf(1,43,2021, "São Paulo","Brasil")
var Array5: IntArray = intArrayOf(7,54,91,70)
```

Operador de acesso ao índice

Aqui estão algumas expressões que usam o operador de acesso de índice com funções correspondentes no Kotlin.

Expressão	Equivalente
<code>a[i]</code>	<code>a.get(i)</code>
<code>a[i, n]</code>	<code>a.get(i, n)</code>
<code>a[i1, i2, ..., in]</code>	<code>a.get(i1, i2, ..., in)</code>
<code>a[i] = b</code>	<code>a.set(i, b)</code>
<code>a[i, n] = b</code>	<code>a.set(i, n, b)</code>
<code>a[i1, i2, ..., in] = b</code>	<code>a.set(i1, i2, ..., in, b)</code>

Exemplos: Operador de acesso ao índice

```
fun main() {  
    val lista = intArrayOf(1,2,3,4,5)  
    for (x in 0..4)  
        print(" " + lista.get(x))  
}
```

ou

```
fun main() {  
    val lista = intArrayOf(1,2,3,4,5)  
    for (x in 0..4)  
        print(" " + lista[x])  
}
```

1 2 3 4 5

1 2 3 4 5

```
fun main() {  
    val lista = intArrayOf(1,2,3,4,5)  
    for (x in 0..4){  
        lista[x] = lista[x]*10  
        print(" " + lista[x])  
    }  
}
```

ou

```
fun main() {  
    val lista = intArrayOf(1,2,3,4,5)  
    for (x in 0..4){  
        lista.set(x, lista[x]*10)  
        print(" " + lista[x])  
    }  
}
```

10 20 30 40 50

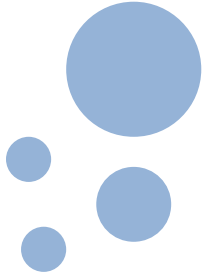
10 20 30 40 50

Classe em Java

- ```
public class Cliente {
 private String email;
 private String nomeUsuario;
 public Cliente(String email, String nomeUsuario) {
 this.email = email;
 this.nomeUsuario = nomeUsuario;
 }
 public String getNomeUsuario() {
 return nomeUsuario;
 }
 public void setNomeUsuario(String nomeUsuario) {
 this.nomeUsuario = nomeUsuario;
 }
 public String getEmail() {
 return email;
 }
 public void setEmail(String email) {
 this.email = email;
 }
}
```



# Classe em Kotlin



- `data class Cliente(var email:String, var nomeUsuario: String)`

# Proteção contra nulos

- Java

- NullPointerException

String nomeUsuario = null;

String nomeUsuario = null;

// Essa linha de código gera um erro de NullPointerException

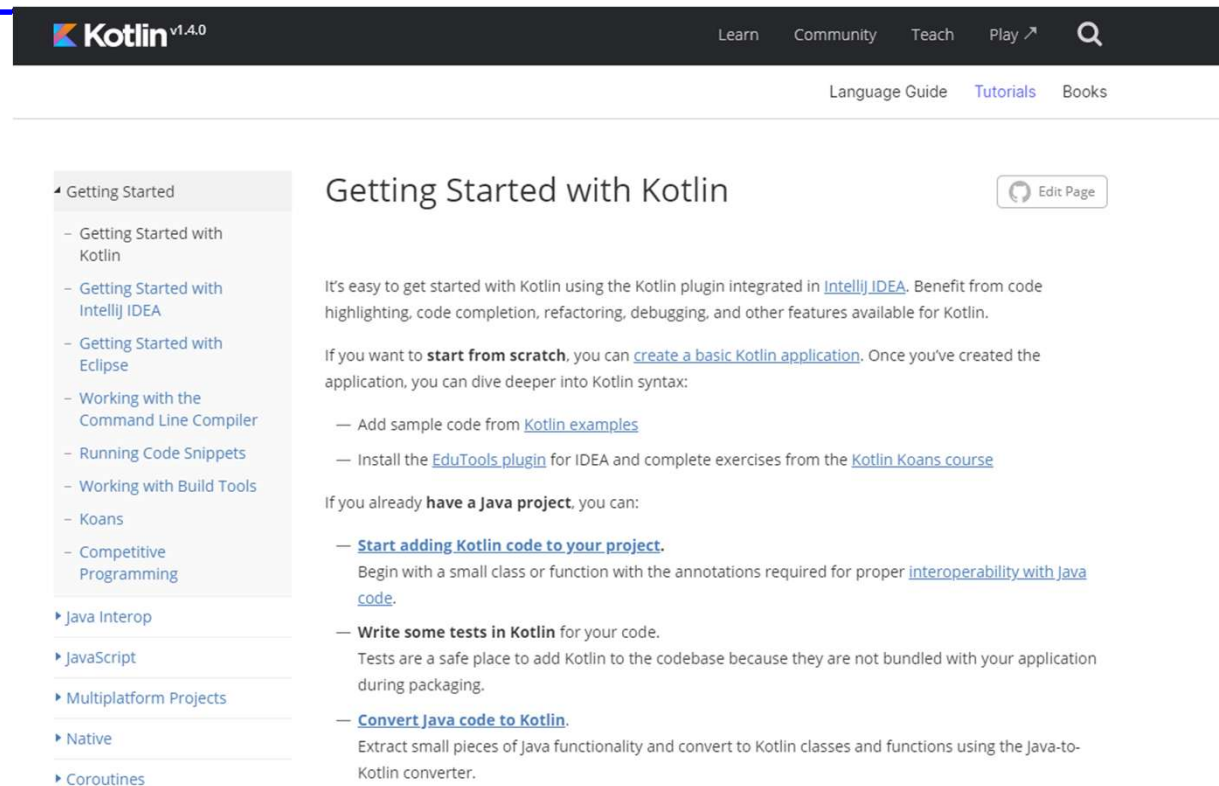
- Kotlin

– var nomeUsuario

:String? = null

# Começando com o Kotlin

- <https://kotlinlang.org/docs/tutorials/getting-started.html>



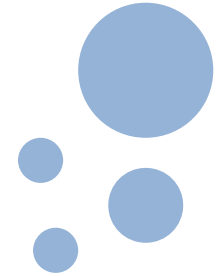
The screenshot shows the Kotlin v1.4.0 documentation page. The header includes the Kotlin logo and version, navigation links (Learn, Community, Teach, Play), and a search icon. Below the header, there are links for 'Language Guide', 'Tutorials', and 'Books'. The main content area is titled 'Getting Started with Kotlin' and includes an 'Edit Page' button. The left sidebar contains a list of navigation items: 'Getting Started' (selected), 'Getting Started with Kotlin', 'Getting Started with IntelliJ IDEA', 'Getting Started with Eclipse', 'Working with the Command Line Compiler', 'Running Code Snippets', 'Working with Build Tools', 'Koans', 'Competitive Programming', 'Java Interop', 'JavaScript', 'Multiplatform Projects', 'Native', and 'Coroutines'. The main text area explains how to get started with Kotlin, mentioning the Kotlin plugin in IntelliJ IDEA, and provides links to 'create a basic Kotlin application', 'Kotlin examples', 'EduTools plugin', 'Kotlin Koans course', 'Start adding Kotlin code to your project', 'interoperability with Java code', 'Write some tests in Kotlin', and 'Convert Java code to Kotlin'.

# Primeiro Programa em Kotlin

- `println("Hello World Fatec")`
- `print ("Hello World Fatec")`
- Ponto virgula não é obrigatório
- Extensão `.kt`

```
fun main() {
 print ("Hello World !!!")
}
```

# Kotlin PlayGround



## Kotlin Playground

Temos uma ferramenta online para aprender e testar os códigos Kotlin, sem a necessidade de criar um projeto no Android Studio para desenvolvimento do aprendizado.

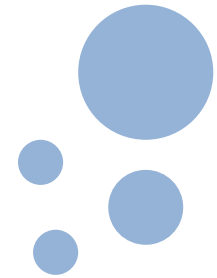
Vamos utilizar esta plataforma para aprender as sintaxes da Linguagem Kotlin e depois realizaremos a criação de aplicativos com o Android Studio.

Link da plataforma: <https://developer.android.com/training/kotlinplayground>



# Try Kotlin

<https://kotlinlang.org/>



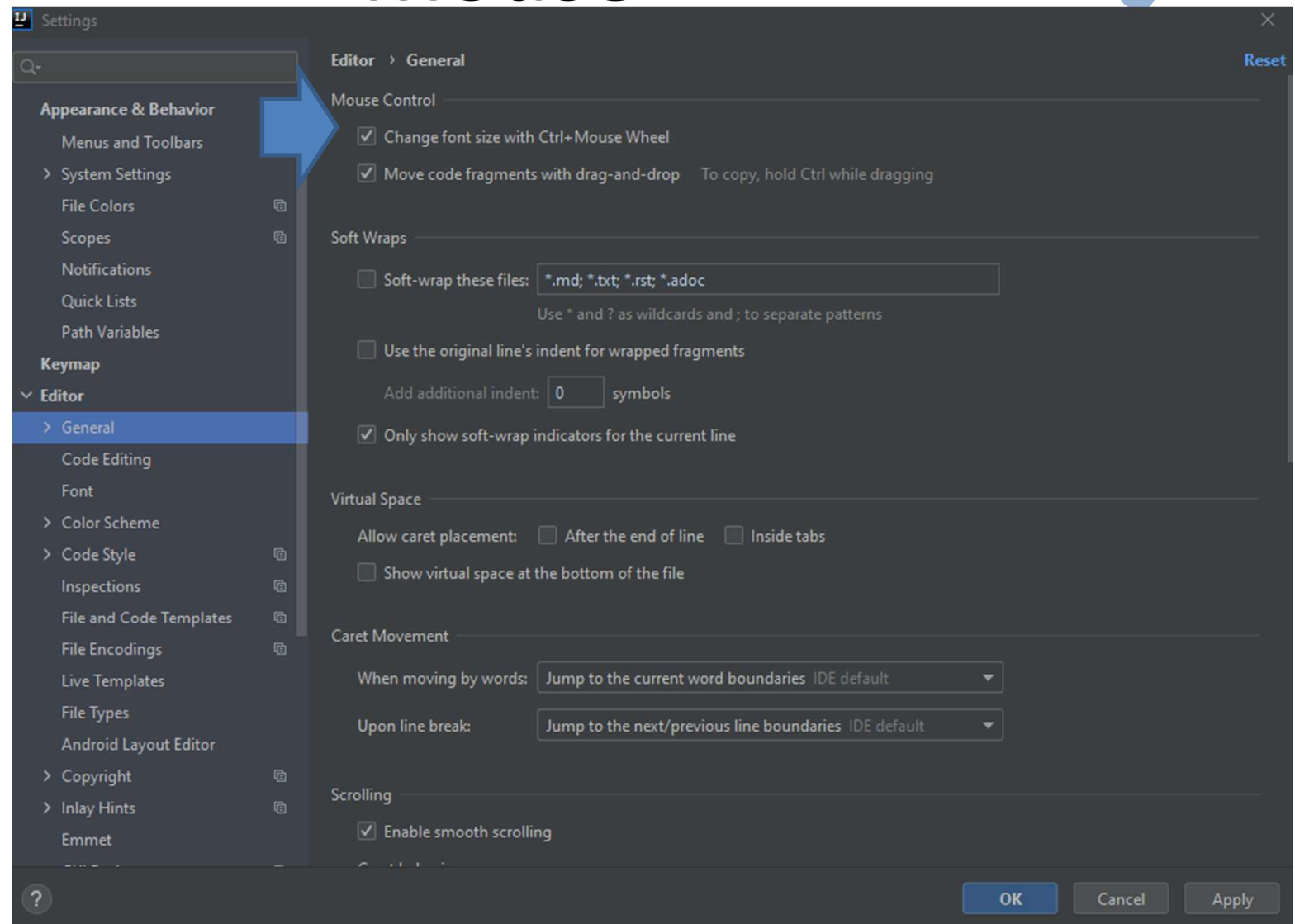
Try Kotlin

Simplest version An Object-oriented Hello Coroutines More examples ↗

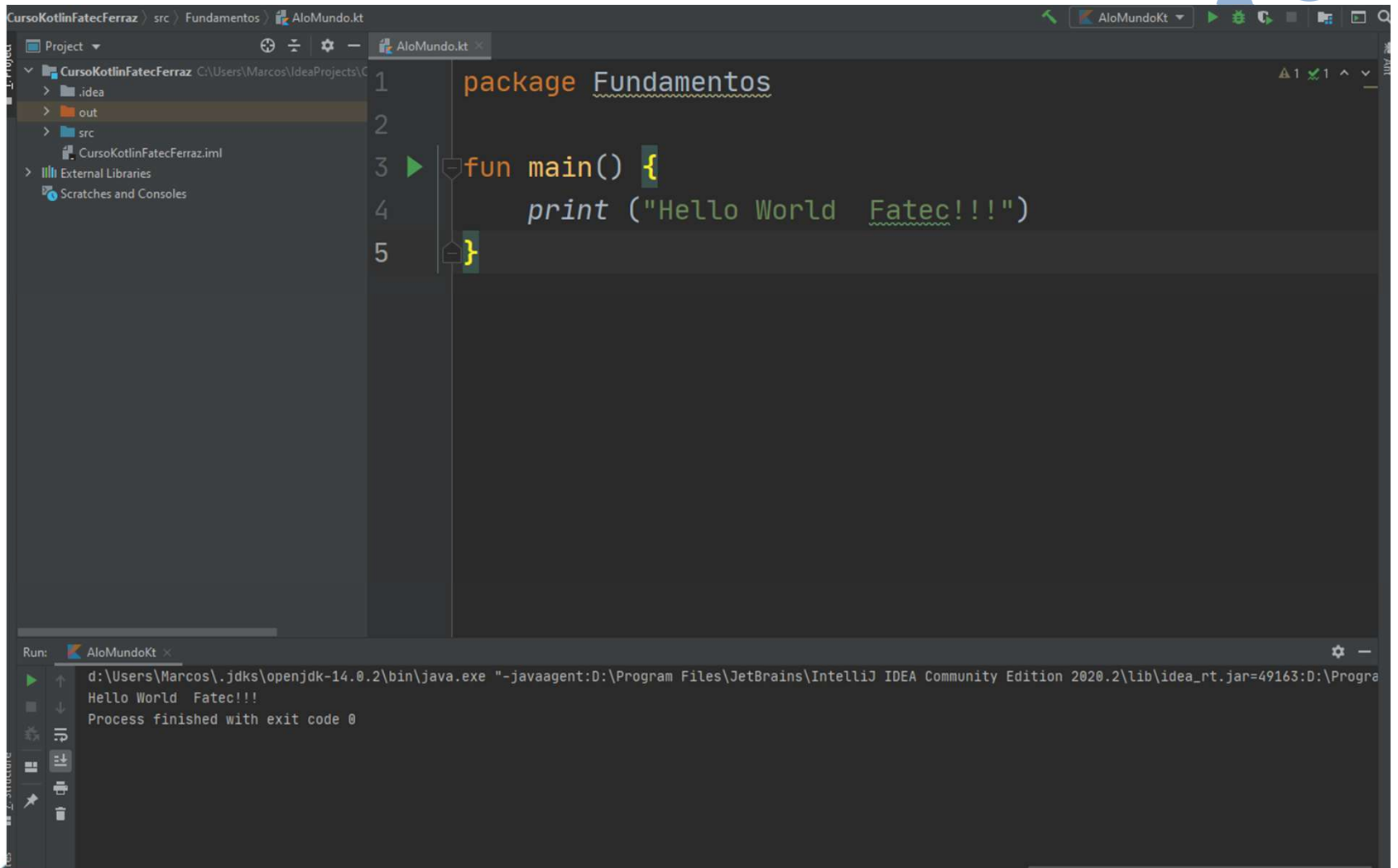
```
fun main()
{
 print ("Hello World !!!");
}
```

Hello World !!!

# Aumentar a fonte através do Mouse



# Hello World !!!



The screenshot displays the IntelliJ IDEA IDE interface. The main editor window shows a Kotlin file named `AloMundo.kt` with the following code:

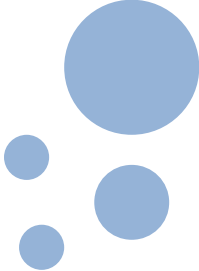
```
1 package Fundamentos
2
3 fun main() {
4 print("Hello World Fatec!!!")
5 }
```

The left sidebar shows the project structure for `CursoKotlinFatecFerraz`, with the `src` directory selected. The bottom panel shows the Run configuration for `AloMundoKt` and the output of the execution:

```
Run: AloMundoKt
d:\Users\Marcos\.jdk\openjdk-14.0.2\bin\java.exe "-javaagent:D:\Program Files\JetBrains\IntelliJ IDEA Community Edition 2020.2\lib\idea_rt.jar=49163:D:\Progra
Hello World Fatec!!!
Process finished with exit code 0
```



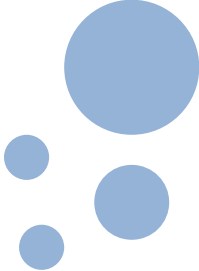
# Hello World !!!



```
fun main() {
 print ("Hello World Fatec!!!")
}
```



# Hello World !!!



```
1. fun main()
2. {
3. print ("Hello World Fatec Ferraz!!!")

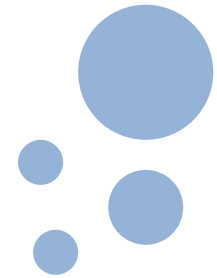
4. }
```

# Comentários

```
1 package Fundamentos
2 fun main()
3 {
4 // Comentário de uma linha
5
6 /*
7 Comentário de várias linhas
8 */
9
10 print ("Hello World Fatec Ferraz!!!")
11
12 }
13
```

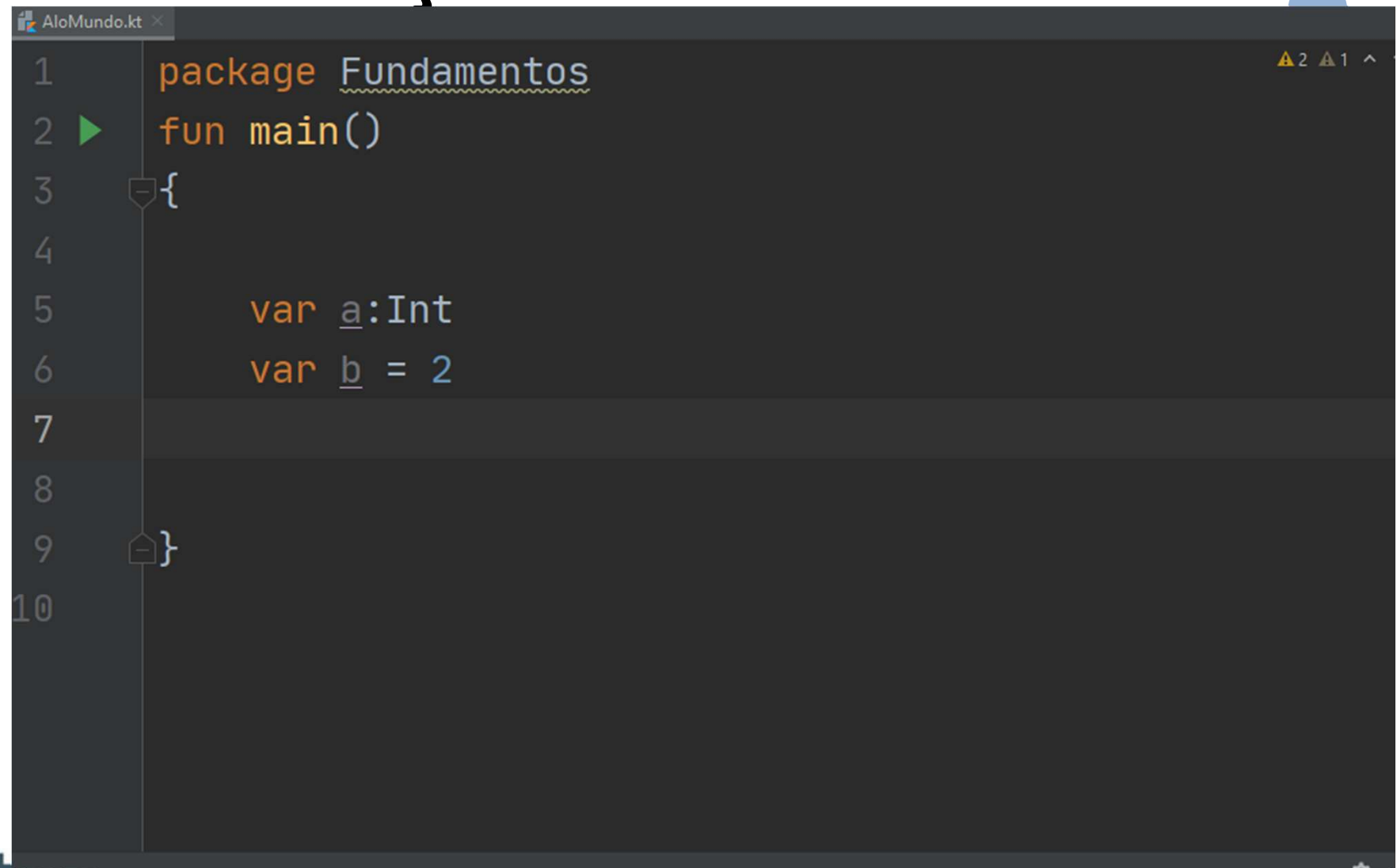


# IntelliJ Idea



- Ctrl + barra de espaço = auto complete

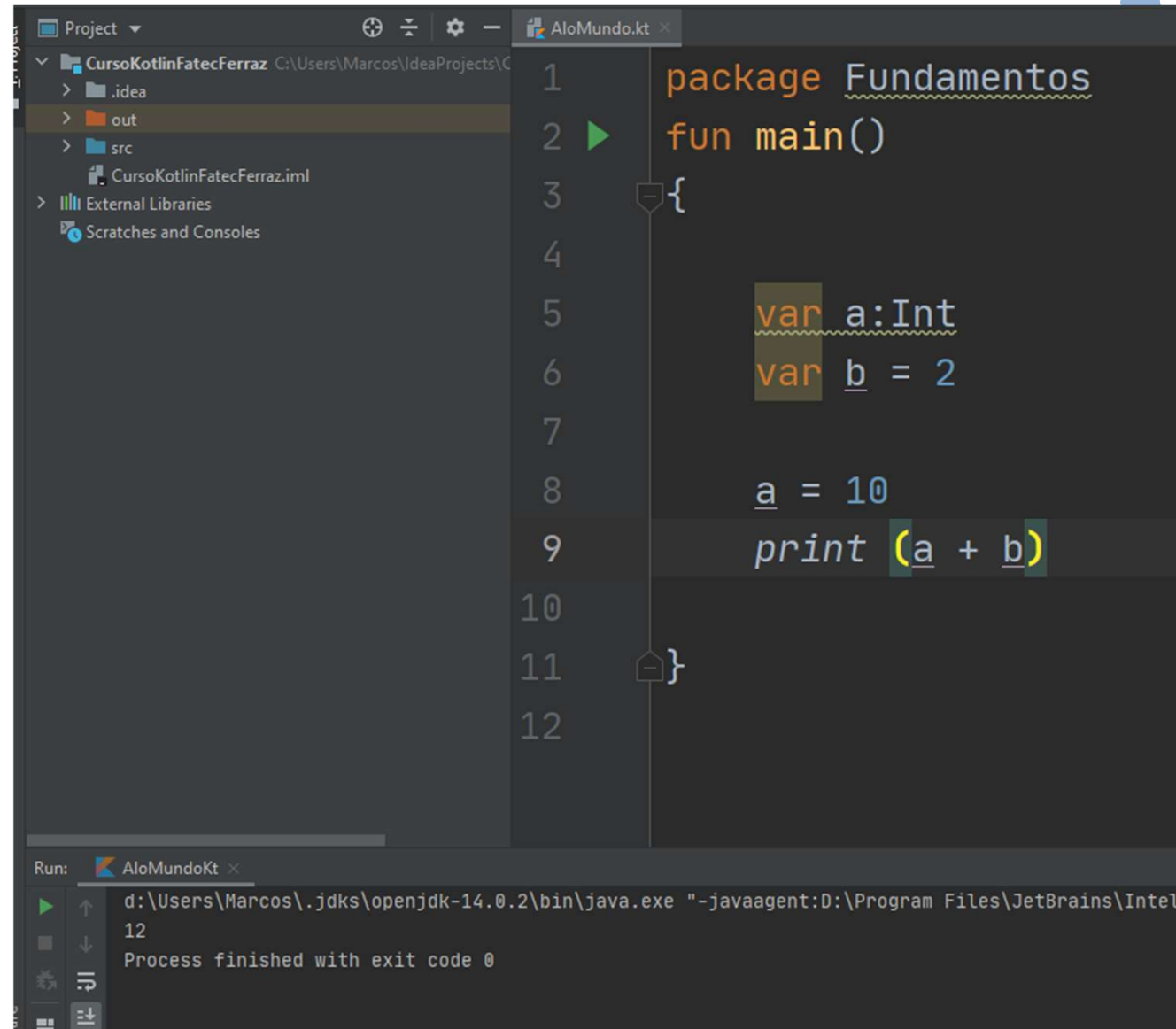
# Criação de variáveis



The screenshot shows a code editor window titled 'AloMundo.kt'. The code is written in Kotlin and defines a package 'Fundamentos' and a main function. Inside the main function, two integer variables are declared: 'a' and 'b'. Variable 'a' is declared with the type 'Int', and variable 'b' is declared and assigned the value 2. The code is as follows:

```
1 package Fundamentos
2 fun main()
3 {
4
5 var a: Int
6 var b = 2
7
8
9 }
10
```

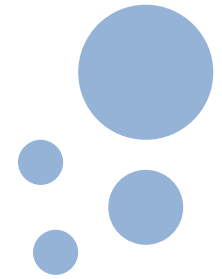
# Criação de variáveis



The screenshot shows an IDE window with a project named 'CursoKotlinFatecFerraz'. The file 'AloMundo.kt' is open, displaying the following Kotlin code:

```
1 package Fundamentos
2 fun main()
3 {
4
5 var a: Int
6 var b = 2
7
8 a = 10
9 print(a + b)
10
11 }
12
```

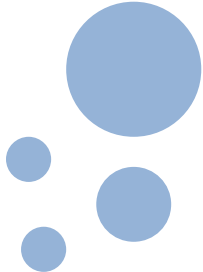
The code is executed, and the Run window at the bottom shows the command: `d:\Users\Marcos\.jdk\openjdk-14.0.2\bin\java.exe "-javaagent:D:\Program Files\JetBrains\IntelliJ IDEA\bin\idea_rt.jar=14770:D:\Program Files\JetBrains\IntelliJ IDEA\bin" d:\Users\Marcos\IdeaProjects\CursoKotlinFatecFerraz\src\main\kotlin\AloMundo.kt`. The output is `12`, and the message 'Process finished with exit code 0' is displayed.



- Variáveis no Kotlin – fortemente tipada.
- Ctrl + shift + f10 = rodar o programa



# Criação de variáveis

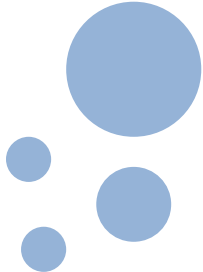


```
1. fun main()
2. {
3. var a: Double
4. var b:Int = 3

5. a = 10.0
6. print (a + b)
7. }
```



# Constantes



```
1. fun main()
2. {

3. val a: Int = 5
4. var b = 3
5. val c: Int
6. c = 10
7. print (a + b + c)
```

# Funções

```
fun calcular(a: Int, b: Int = 1): Int
```

Palavra-chave

Nome

Parâmetro

Argumento Padrão  
(Opcional)

Tipo de Retorno

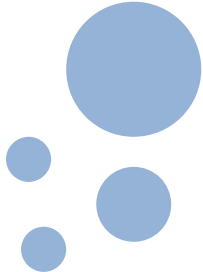
## Interoperabilidade Java

```
println(Math.PI)
```

Chamando bibliotecas Java nativamente sem wrappers ou configurações extras.



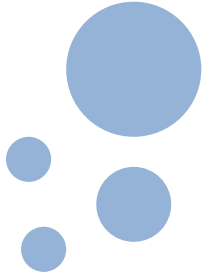
# Constantes



```
1. fun main()
2. {
3. val a: Int = 5
4. var b = 3
5. val c: Int
6. c = 10
7. println (a + b + c)
8. println (Math.PI) //chamando do JAVA
9. }
```



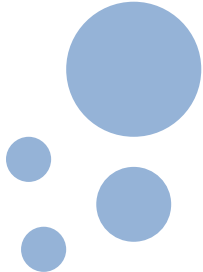
# Tipos de Dados



- Tipos Básicos do Kotlin
- Byte: 8 bits (1 byte)
- Short: 2 bytes
- Int: 4 bytes
- Long: 8 bytes

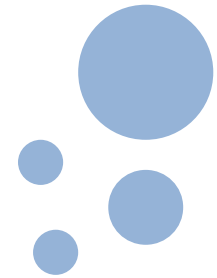


# Tipos de Dados



```
1. fun main()
2. {
3. var n1: Byte = 127
4. var n2: Short = 32767
5. var n3: Int = 2_147_483_647
6. var n4: Long = 9_223_372_036_854_775_807
7. println("Fim do Programa")
8. }
```

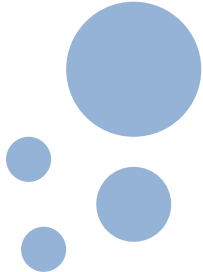
# Tipos de Dados



```
1. fun main()
2. {
3. var n1: Byte = 127
4. var n2: Short = 32767
5. var n3: Int = 2_147_483_647
6. var n4: Long = 9_223_372_036_854_775_807
7. println(Long.MAX_VALUE)
8. println("Fim do Programa")
9. }
```



# Tipos de Dados



```
1. fun main()
2. {
3. var n1: Float = 3.14F
4. var n2: Double = 3.14

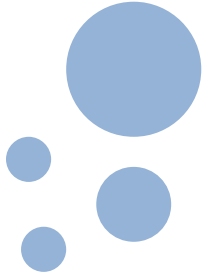
5. println(Float.MAX_VALUE)
6. println(Double.MAX_VALUE)
7. println("Fim do Programa")
8. }
```

# Tipos de Dados

1. fun main()
2. { //String
3.     var nome: String = "Fatec"
4.     //Tipo Caractere
5.     var c: Char = 'M'
6.     //Tipo Booleano
7.     var b:Boolean = true //ou false  
            não aceita (0 ou 1)
8.     println("Fim do Programa")



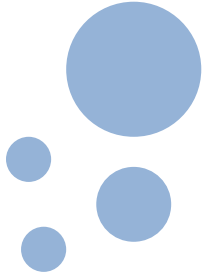
# Tipos de Dados



```
1. fun main()
2. {
3. //Tipo Caractere
4. var c: Char = 'M'
5. //Tipo Booleano
6. var b:Boolean = true // ou false
7. println (2 is Int)
8. println (2.4 is Double)
9. println("Fim do Programa")
10. }
```



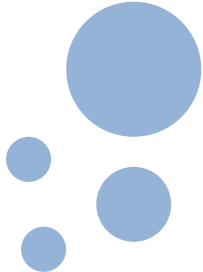
# Tipos de Dados



- Tudo no Kotlin é objeto

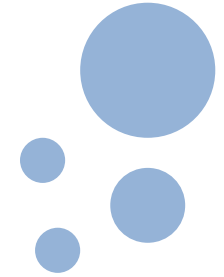


# Tipos de Dados



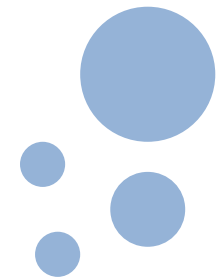
```
1. fun main()
2. {
3. soma (5,7)
4. }
5. fun soma(a: Int, b: Int)
6. {
7. println(a+b)
8. }
```

# Tipos de Dados



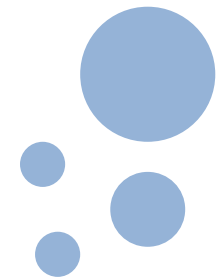
```
fun main()
{
 println(soma (5,7))
 println(soma(5))
}
fun soma(a: Int, b: Int = 1): Int
{
 return a+b
}
```

# Tipos de Dados



```
fun main()
{
 val aprovados = listOf("João", "Maria", "Pedro")
 println("O primeiro colocado foi ${aprovados[0]}")
}
```

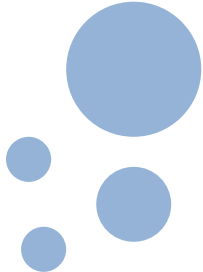
# Tipos de Dados



```
fun main()
{
 val aprovados = listOf("João", "Maria", "Pedro")
 println("O primeiro colocado foi " + aprovados[0])
}
```



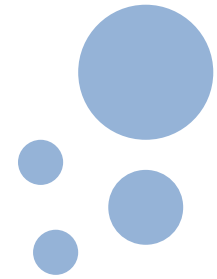
# Safe call operator



- No Kotlin significa que a variável pode ter valor nulo
- `var a:Int? = null`

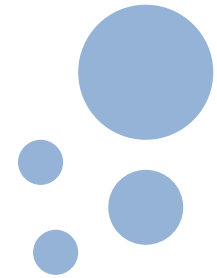
```
package Fundamentos
fun main()
{
 var a:Int? = null
 println (a)
}
```

# Operador Elvis (?:)



- Operador Elvis (?:)

```
fun main()
{
 var a:String? = null
 val b: String = a ?: "Teste"
 println (b)
}
```



```
fun main()
{
 var a:String? = null

 println (a!!)
}
```

```
Exception in thread "main" kotlin.KotlinNullPointerException
 at Fundamentos.AloMundoKt.main(AloMundo.kt:6)
 at Fundamentos.AloMundoKt.main(AloMundo.kt)
```

# Operadores

- Atribuição

```
fun main()
{
 var a:Int = 7
 var b:Int = 3

 b += a // b = b + a
 println(b)

 b -= 4 // b = b - 4
 println(b)
}
```

```
fun main()
{
 var a:Int = 7
 var b:Int = 3

 b += a // b = b + a
 println(b)

 b -= 4 // b = b - 4
 println(b)

 b *= 2 // b = b * 2
 println(b)

 b /= 2 // b = b / 2
 println(b)

 b %= 2 // b = b % 2
 println(b)
}
```

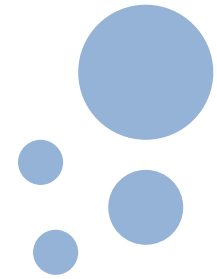
# Operadores

- Aritméticos

```
fun main()
{
 var a:Int = 7
 var b:Int = 3

 val soma = a + b
 val subtracao = a - b
 val multiplicacao = a * b
 val divisao = a / b
 val modulo = a % 2

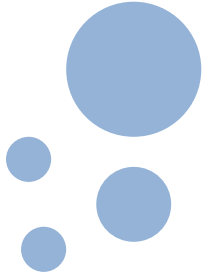
 println("$soma $subtracao $multiplicacao $divisao $modulo")
}
```



- Ctrl + alt + I = reformatar o código



# Operadores

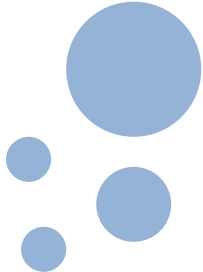


- Relacionais

```
fun main() {
 println(1 == 1)
 println(3 != 2)
 println(3 > 2)
 println(3 < 2)
 println(3 <= 2)
 println(3 >= 2)
}
```



# Operadores



## Lógicos

```
fun main() {
 val a = true
 val b = false

 val r1 = a || b
 val r2 = a && b
 val r3 = a xor b
 print("$r1 $r2 $r3")
}
```

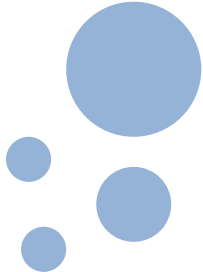
```
fun main() {
 var a = true

 a = !a

 print("$a")
}
```



# Operadores



- Incremento/Decremento

```
fun main() {
 var a = 1

 a++
 println("$a")

 a--
 println("$a")
}
```

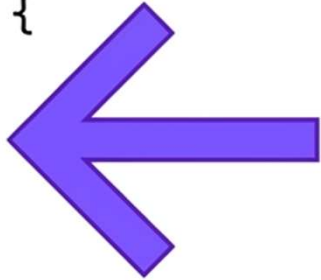
# Controle de Fluxo: If como Expressão

Em Kotlin, 'if' retorna um valor. Adeus operador ternário!

```
val nota = 7.0

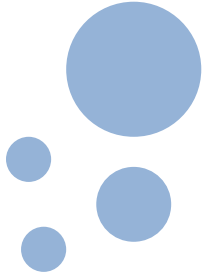
val resultado = if (nota >= 6.0) {
 "Aprovado"
} else {
 "Reprovado"
}

println(resultado) // Imprime: Aprovado
```





# Operadores



- Simulando operador ternário

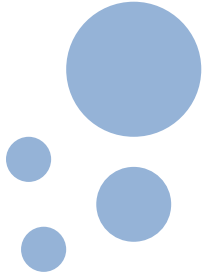
```
fun main() {
 val nota: Double = 5.0

 val resultado: String = if (nota >= 6) "Aprovado" else "Reprovado"

 println(resultado)
}
```



# Conversão

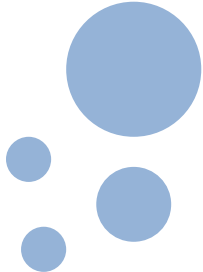


- Número para String

```
fun main() {
 val a = 1
 //Converte Número para String
 println(a.toString() + 1)
}
```



# Conversão



- Número para String

```
fun main() {

 //String para número
 println("1.9".toDouble() + 3)

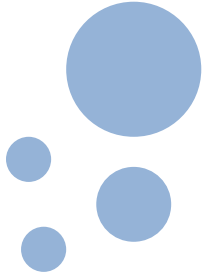
}
```

# Checagem de Tipos

```
fun main() {
 val valor: String = "Fatec"

 if (valor is String) {
 println(valor)
 } else if (valor !is String) {
 println("Não é uma String")
 }

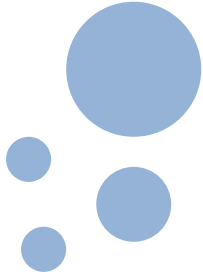
}
```



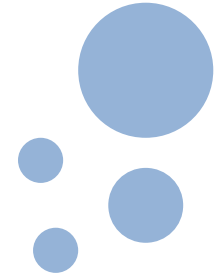
```
fun main() {
 //var a: Int = 10
 //var b: Int = 20
 var a: Int
 var b: Int
 println("Digite o primeiro número")
 a = readLine().toString().toInt()
 println("Digite o primeiro número")
 b = readLine().toString().toInt()
 var multiplica: Int
 multiplica = a * b

 print("O resultado da Multiplicação é $multiplica")

}
```



```
//Nome: Marcos Vasconcelos de Oliveira
//RA: 10101020
//Objetivo: 03. Faça um programa na Linguagem Kotlin que
// receba 2 números e apresente a multiplicação dos
// dois números
//fases para programação
//1a. Fase - Entender o Problema
//2a. Fase - Detalhar o Problema
// Passo a Passo - recebe o número/faça o calculo/xx
// E/P/S
// E ==> n1, n2
// P ==> multiplica = n1 * n2
// S ==> multiplica
//3a. Fase = fluxograma/ Diagrama de Blocos
//4a. Fase = Testes / Teste de Mesa
//5a. Fase = Codificação = Português Estruturado/Linguagem
```



- <https://repl.it/repls/ShockedWeightyComputeRgame#main.kt>

## Por Que Kotlin?

### Verbosidade do Java

```
public class Cliente {
 private String email;
 private String nomeUsuario;
 public String getNomeUsuario() { return nomeUsuario; }
 public void setNomeUsuario(String nomeUsuario) {
 this.nomeUsuario = nomeUsuario;
 }
 public String getEmail() { return email; }
 public void setEmail(String email) { this.email = email; }
}
```

### Concisão do Kotlin

```
data class Cliente(var email: String,
 var nomeUsuario: String)
```



### Concisão

Redução drástica de código repetitivo.



### Segurança

Proteção nativa contra NullPointerException.



### Interoperabilidade

100% compatível com Java (JVM e Bitcode).

# Variáveis e Mutabilidade



## val (Value)

Imutável / Constante  
(Recomendado)

```
val pi = 3.14
```

Uma vez atribuído, o valor não pode mudar.



## var (Variable)

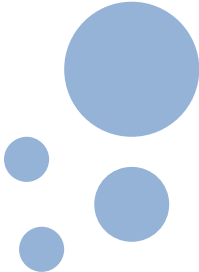
Mutável

```
var idade = 25
idade = 26
```

O valor pode ser alterado posteriormente.

Inferência de Tipo: O Kotlin detecta o tipo automaticamente.

```
var b = 3 // O compilador sabe que é Int
```



# Tipos de Dados Fundamentais

## Números Inteiros

Byte, Short, Int (Padrão), Long

```
val n: Int = 10
val l: Long = 100L
```

## Ponto Flutuante

Float, Double (Padrão)

```
val pi: Double = 3.14
val f: Float = 3.14F
```

## Texto

Char (1 caractere), String (Texto)

```
val letra: Char = 'A'
val nome: String = "Fatec"
```

## Lógica

Boolean

```
val ativo: Boolean = true
```

**Atenção:** Boolean aceita apenas true ou false.  
Não aceita 0 ou 1.



### Non-Null (Padrão)

```
var a: String = "Texto"
```

```
a = null // Erro de compilação! Não compila.
```



### Nullable Types

```
var b: String? = "Texto"
```

```
b = null // Permitido pelo '?'
```

### Tratamento Seguro (Elvis Operator)

**?:**

```
val l = b?.length ?: -1 // Se b não for nulo, retorna o tamanho. Se for nulo, retorna -1.
```

## Resumo e Boas Práticas

- ✓ **"Entry Point"**: Todo programa começa em fun main().
- ✓ **"Imutabilidade"**: Prefira 'val' em vez de 'var' para código mais seguro.
- ✓ **"Tipagem"**: Forte e estática, mas use a inferência de tipo.
- ✓ **"Null Safety"**: Evite NullPointerException usando tipos '?' e safe calls.
- ✓ **"Interoperabilidade"**: Não tenha medo de misturar Java e Kotlin.

**Código mais limpo, desenvolvimento mais rápido.**

# Operadores Aritméticos e Atribuição

## Operadores Aritméticos

- + (Soma)
- (Subtração)
- \* (Multiplicação)
- / (Divisão)
- % (Módulo/Resto)

## Atribuição Composta

```
var b = 10
```

```
b += 5 // b vira 15
```

```
b -= 2 // b vira 13
```

```
b *= 2 // b vira 26
```

## String Templates

```
println("O resultado é: $b")
```

Use \$ para injetar variáveis diretamente nas Strings.

# Resumo

## Kotlin: Primeiros Passos para o Desenvolvimento Android



### Preparação do Ambiente

Instale o IntelliJ IDEA e o Java JDK para compilar bitcode na JVM.



### O Ponto de Entrada

Todo programa Kotlin inicia sua execução através do método fundamental "main".



### Pacotes e Organização

Utilize "Packages" para organizar seus arquivos de código como pastas de sistema.



### Variáveis: val vs var

Use "val" para valores imutáveis e "var" para variáveis que podem mudar.



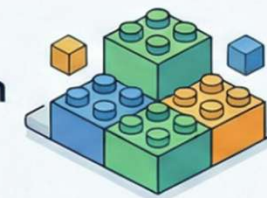
### Segurança contra Nulos

Kotlin protege o código contra erros de "NullPointerException" usando tipos anuláveis com "?".



### Interoperabilidade e Concisão

Kotlin é 100% compatível com Java e foca em código claro e reduzido.



## Conceitos Core do Kotlin

### Resumo dos Tipos de Dados Básicos

| Tipo             | Exemplo / Uso                          |
|------------------|----------------------------------------|
| # Int / Long     | Números inteiros (ex: 2_147_483_647)   |
| o Double / Float | Números decimais ou de ponto flutuante |
| ✓ Boolean        | Valores lógicos: true ou false         |